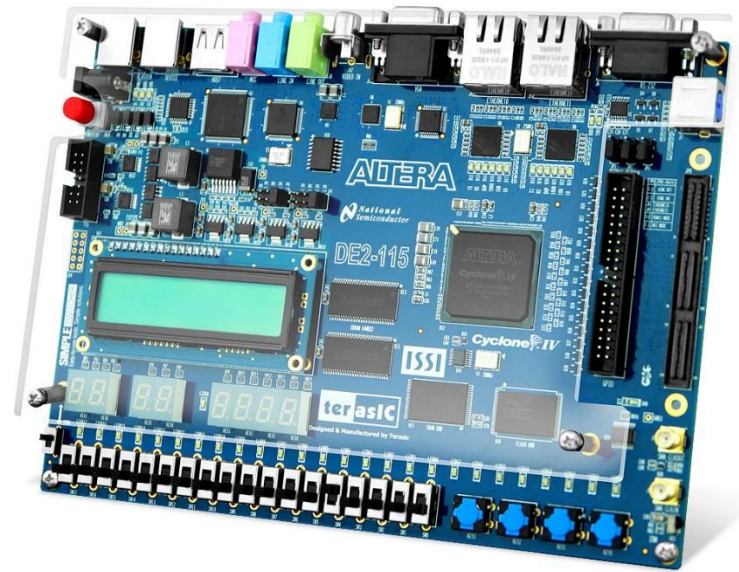


COEN-230

Digital System Design I

Dr. Fadi Alzhouri

Department of Electrical and Computer Engineering
Gust University





Outline

Topics (Ch. 6):

- Synchronous Sequential Circuits
- Design techniques for circuits that use FFs
 - Concept of states and implementation with FFs
 - Sequential behavior of digital circuits
 - Procedure for design and analysis of synchronous sequential circuits

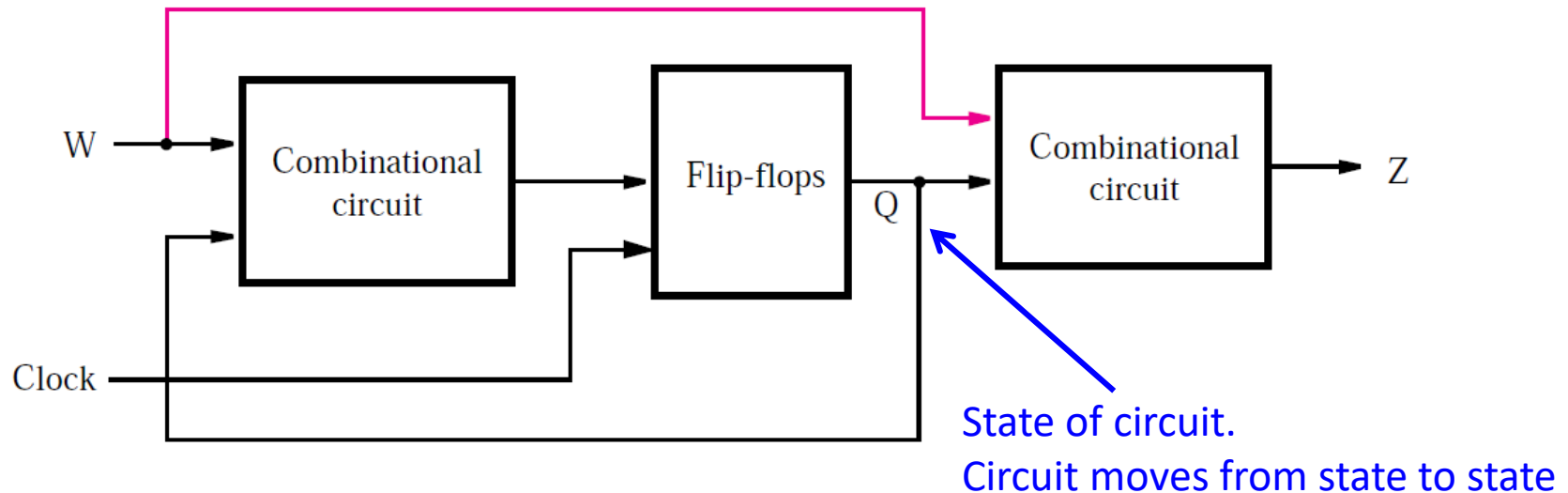


What are sequential circuits?

A circuit whose output(s) depend on past behavior, and present inputs

- Clock is used => synchronous sequential circuits
- No clock => asynchronous sequential circuits

They are realized with combinational logic and one or more flip-flops. Also called *Finite State Machines* (FSM)



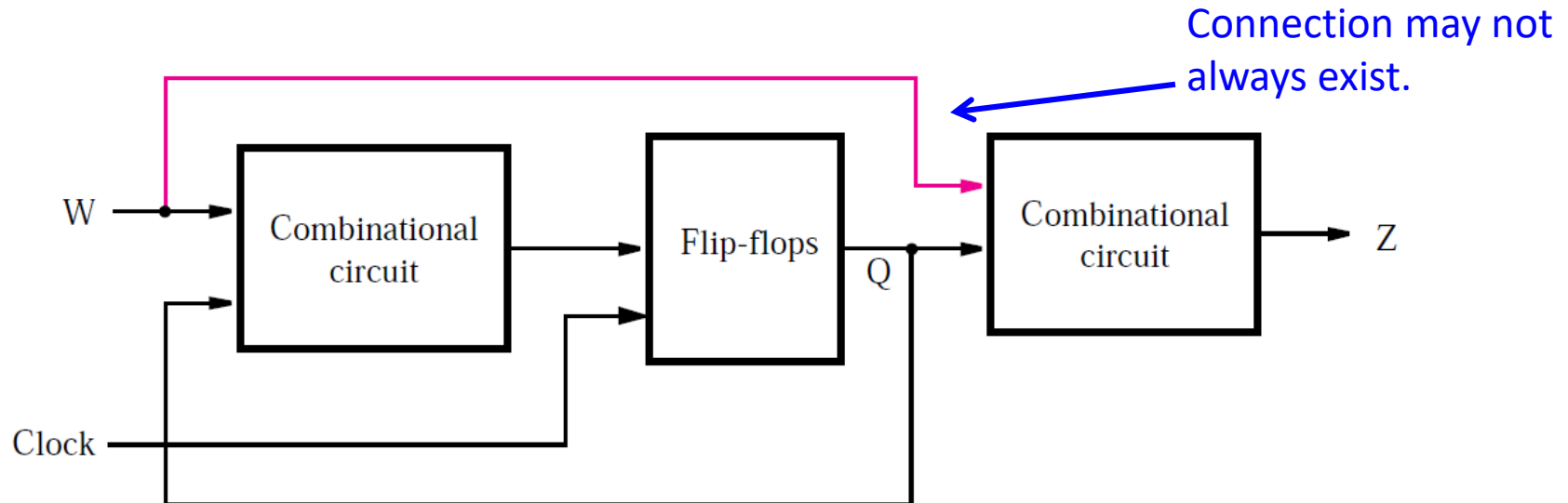


Synchronous Sequential Circuits

State elements in synchronous sequential circuits are **edge** triggered
To ensure state changes only once in a single cycle.

Synchronous sequential circuits are of two types:

- **Moore**: output depends only on state
- **Mealy**: output depends on state and inputs





Applications of FSM

Finite-state machines can solve a large number of problems, among which are:

- electronic design automation
- communication protocol design
- string matching, parsing and other engineering applications
- In biology and artificial intelligence research, state machines or hierarchies of state machines are sometimes used to describe neurological systems and in linguistics—to describe the grammars of natural languages.
- lexical analyzers
- Control systems
- Software specification and design



Basic Design Steps

Suppose that we want a circuit with the following specification:

- One input w , and one output z
- Positive-edge-triggered design
- $z = 1$, if $w = 1$ during **two** preceding clock cycles
- Note: output z cannot depend solely on the present value of w .
Hence circuit must maintain state

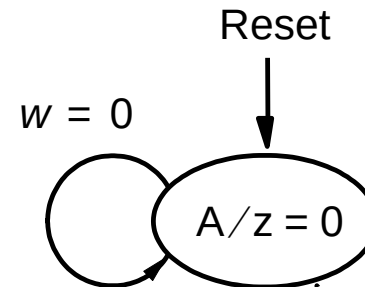
Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	1	0



1. State Diagram

Pictorial representation of a FSM:

- How many states
States are circles
- Transitions between states
Transitions are directed edges
- Starting state
i.e. after reset/clear



Note in figure, reset is not treated as input

- To simplify figure

States will be represented by FFs.
How many?



2. State Table

Another way to describe a FSM. More convenient from an implementation point of view.

It contains information on:

- States of the machine
 - All transitions from **present state** to **next state** for all possible inputs
 - Output values
 - Reset information ignored
- State A is assumed to be “start” state

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1



3. Finding State Assignment

Find number of flip/flops needed to represent state

m FFs can represent at most 2^m states.

How many are needed in previous example?

Assign each state a combination of values of **state variables**

Each state variable may be implemented by a flip-flop.

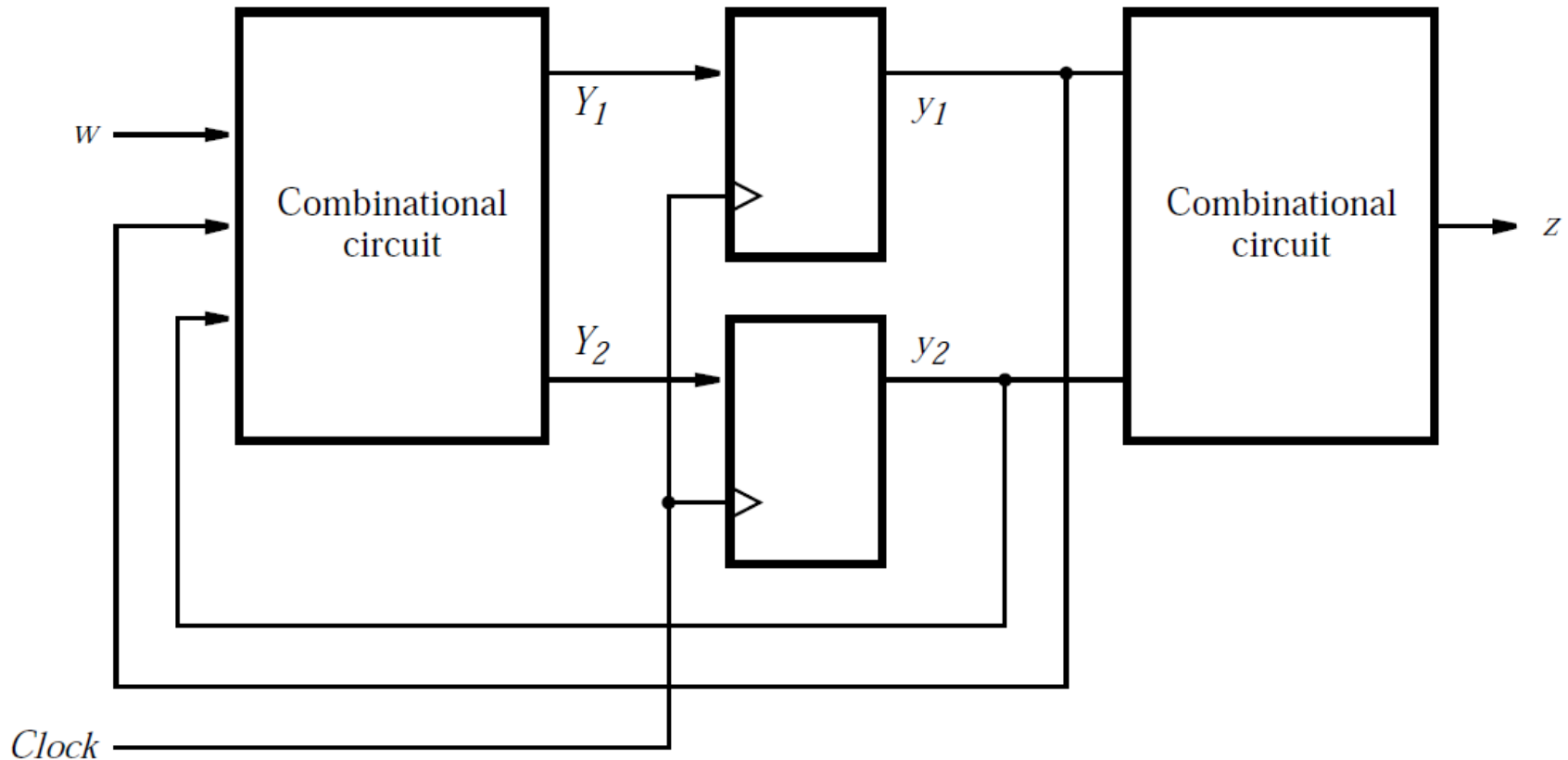
“State assigned table”

All unused variable combinations are normally treated as don't cares

We will discuss more on the topic of state assignment in a while



State variables



Next step is to create a truth table that defines combinatorial circuits



State Assigned Table

Below is the resulting table after state assignment
Notice that:

Output depends on current state only → Moore type

2 state variables are sufficient to represent 3 states

Y_1 & Y_2 are *next-state variables*, y_1 & y_2 are *present-state variables*

We assign a specific valuation to y_1 & y_2

	Present state $y_2 y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d



4. FF Selection & Function Derivation

Need to decide type of FF to use as state element

For now, use D-FF since it is easiest

$D1 = Y1$, and $D2 = Y2$

Other types of FFs require more work!

For every next state and output, derive their function from present state and input

Draw truth table from state table

	Present state $y_2 y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

This state never occurs!



Function Realization

		y_2y_1			
		00	01	11	10
w	0	0	0	d	0
	1	1	0	d	0

Ignoring don't cares

$$Y_1 = w\bar{y}_1\bar{y}_2$$

Using don't cares

$$Y_1 = w\bar{y}_1\bar{y}_2$$

		y_2y_1			
		00	01	11	10
w	0	0	0	d	0
	1	0	1	d	1

$$Y_2 = wy_1\bar{y}_2 + w\bar{y}_1y_2$$

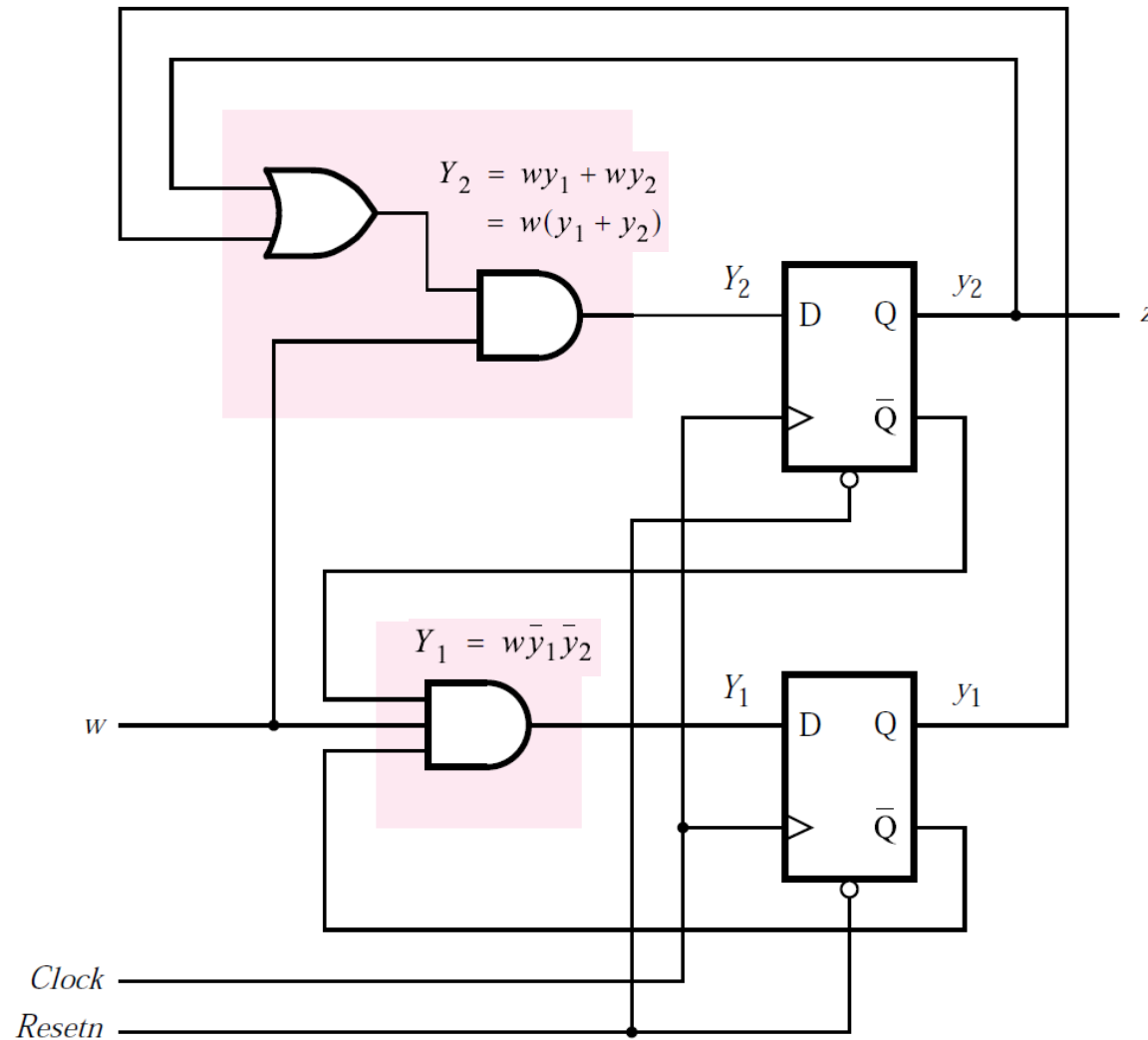
$$\begin{aligned} Y_2 &= wy_1 + wy_2 \\ &= w(y_1 + y_2) \end{aligned}$$

y_2	y_1	
	0	1
0	0	0
1	1	d

$$z = \bar{y}_1y_2$$

$$z = y_2$$

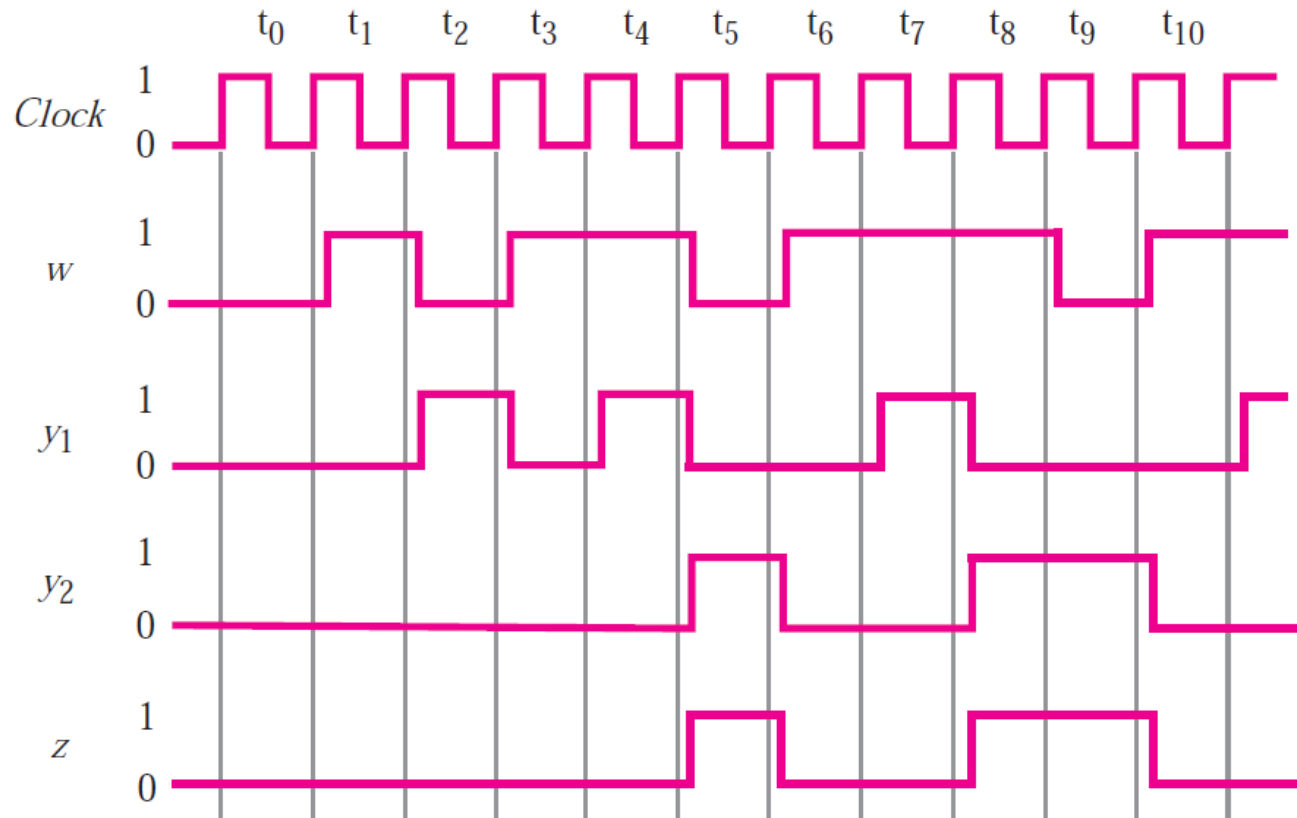
Resulting Logic Circuit





Timing Diagram of Realization

All changes in the signals occur shortly after the positive edge of the clock.





Summary of Design Steps

To design a synchronous sequential circuit:

1. Use the specification to design the state diagram
2. Create a state table
3. Perform state minimization (more on this later)
4. Perform state assignment
5. Choose FFs (D, SR, JK, and/or T)
6. Derive the next-state & output functions needed to implement the circuit



State Assignment Problem

State assignment has direct relation to the cost of derived implementation

Some state assignments are better than others

Example from before:

	Present state $y_2 y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

Previous assignment

	Present state $y_2 y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

New assignment



Improved state assignment

- Using the new state assignment a more cost effective realization is possible

	Present state $y_2 y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

	Present state $y_2 y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

$$Y_1 = w \cdot \bar{y}_1 \cdot \bar{y}_2$$

$$Y_2 = w(y_1 + y_2)$$

$$z = y_2$$

$$Y_1 = w, \text{ cheaper}$$

$$Y_2 = w y_1, \text{ cheaper}$$

$$z = y_2, \text{ same cost}$$

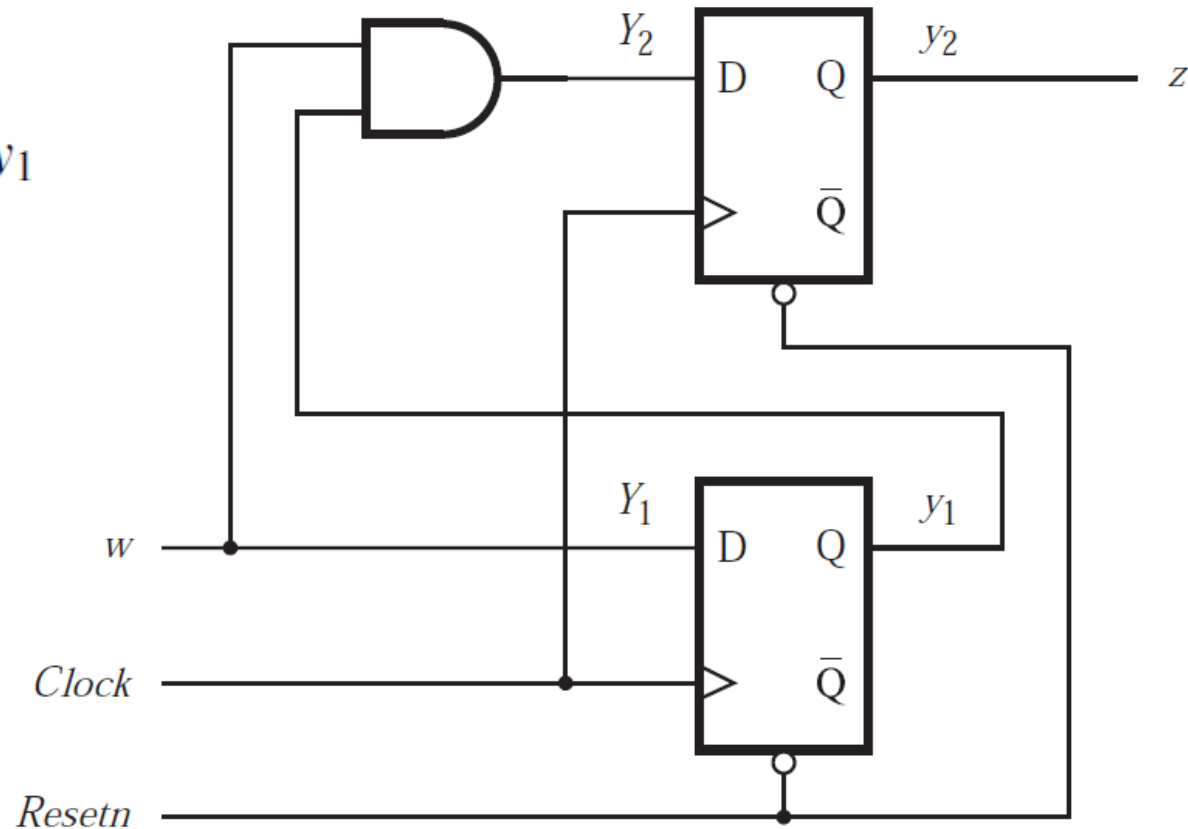


Improved circuit

$$Y_1 = D_1 = w$$

$$Y_2 = D_2 = wy_1$$

$$z = y_2$$





One-hot encoding

As an alternative state assignment, we can use **one-hot encoding**:

Use as many state variables as there are states in circuit

For each state, all-but-one of state variables are equal to zero (one-hot)

Usually leads to simpler output expressions, faster circuits

However, in this example, it is no cheaper than previous example

Do this as an exercise

	Present state	Next state		Output z
		$w = 0$	$w = 1$	
	$y_2 y_1$	$Y_2 Y_1$	$Y_2 Y_1$	
A	00	00	01	0
B	01	00	10	0
C	10	00	10	1
	11	dd	dd	d

	Present state	Next state		Output z
		$w = 0$	$w = 1$	
	$y_3 y_2 y_1$	$Y_3 Y_2 Y_1$	$Y_3 Y_2 Y_1$	
A	0 0 1	0 0 1	0 1 0	0
B	0 1 0	0 0 1	1 0 0	0
C	1 0 0	0 0 1	1 0 0	1

Remaining five are 'don't cares'



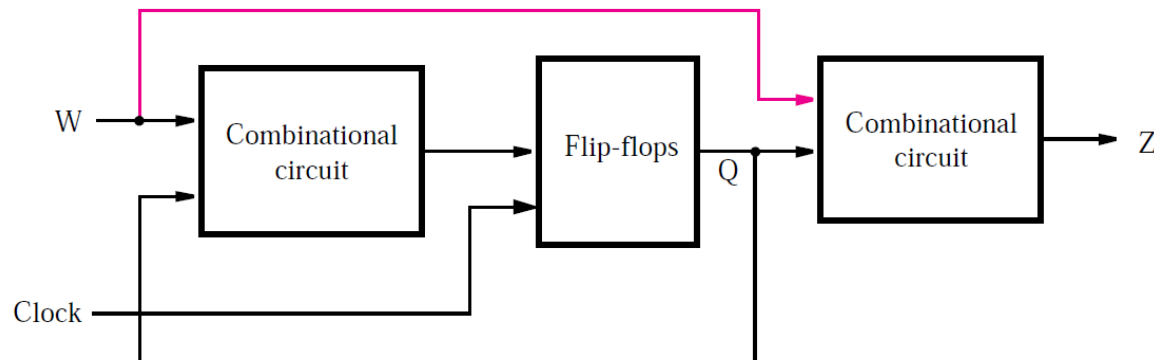
Mealy State Model

Previous examples were sequential circuits in which each state had specific values of the output signals associated with it.

Such finite state machines are said to be of **Moore** type.

In **Mealy-type** machines the output values are generated based on both the state of the circuit **and** the present values of its inputs.

This provides additional flexibility in the design of sequential circuits.

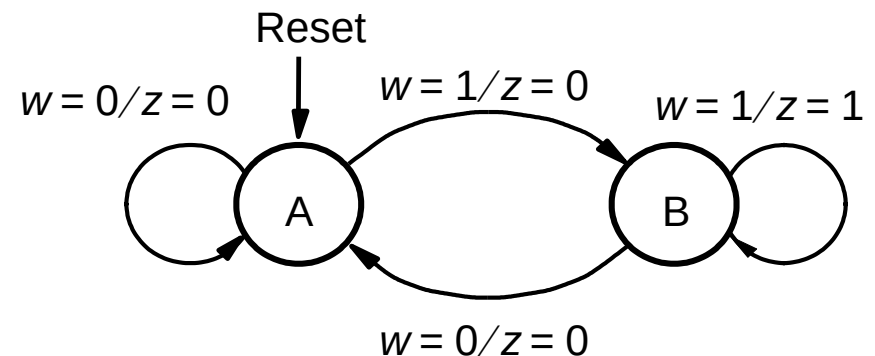
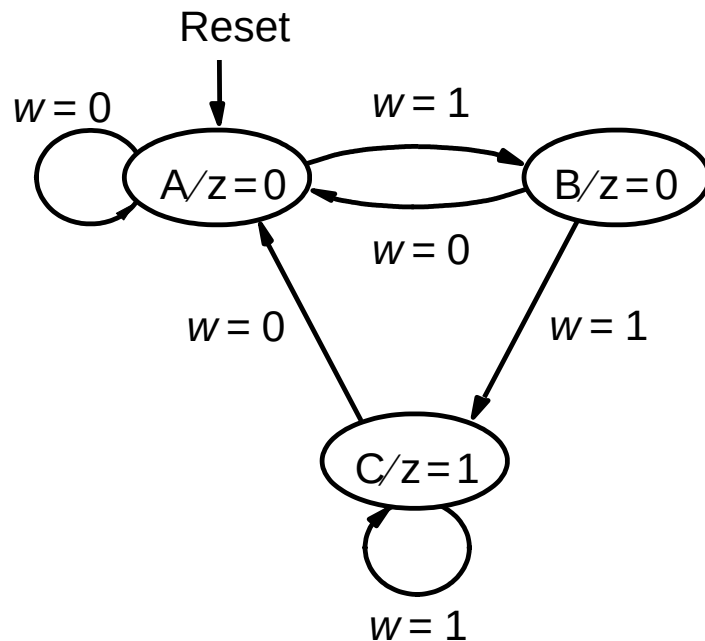




Mealy State Model

Assume the previous example is modified so that output is 1 in the same clock cycle when the second $w=1$ appears.

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	1	0	0	1	1	0	0





Mealy State Table

Note how output is a function of state and input

Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

	Present state	Next state		Output	
		$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

Mealy FSM



Mealy Machine Implementation

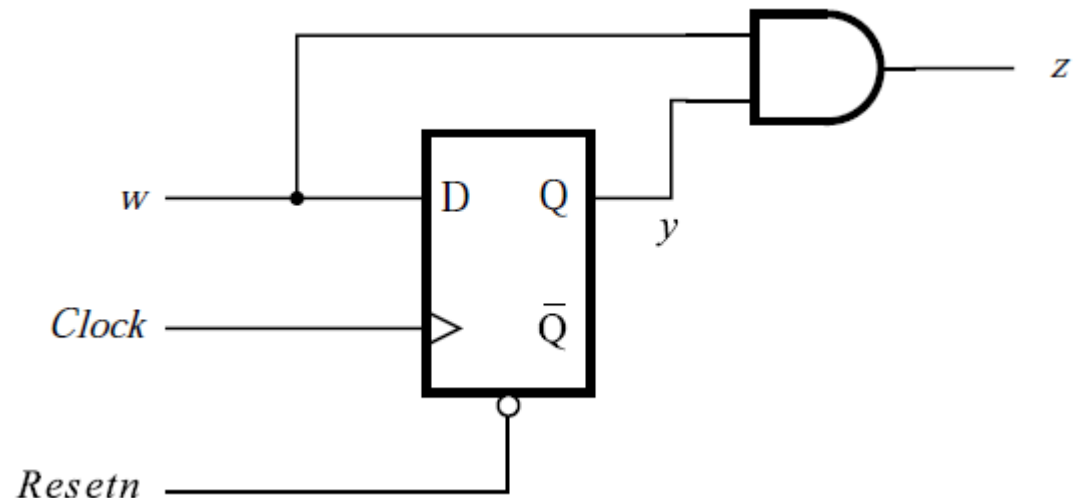
Z is high when y & w equal to 1

Note that mealy implementation is more cost effective than Moore implementation

However, circuit can be modified so that it behaves like a Moore machine

$$Y = D = w$$

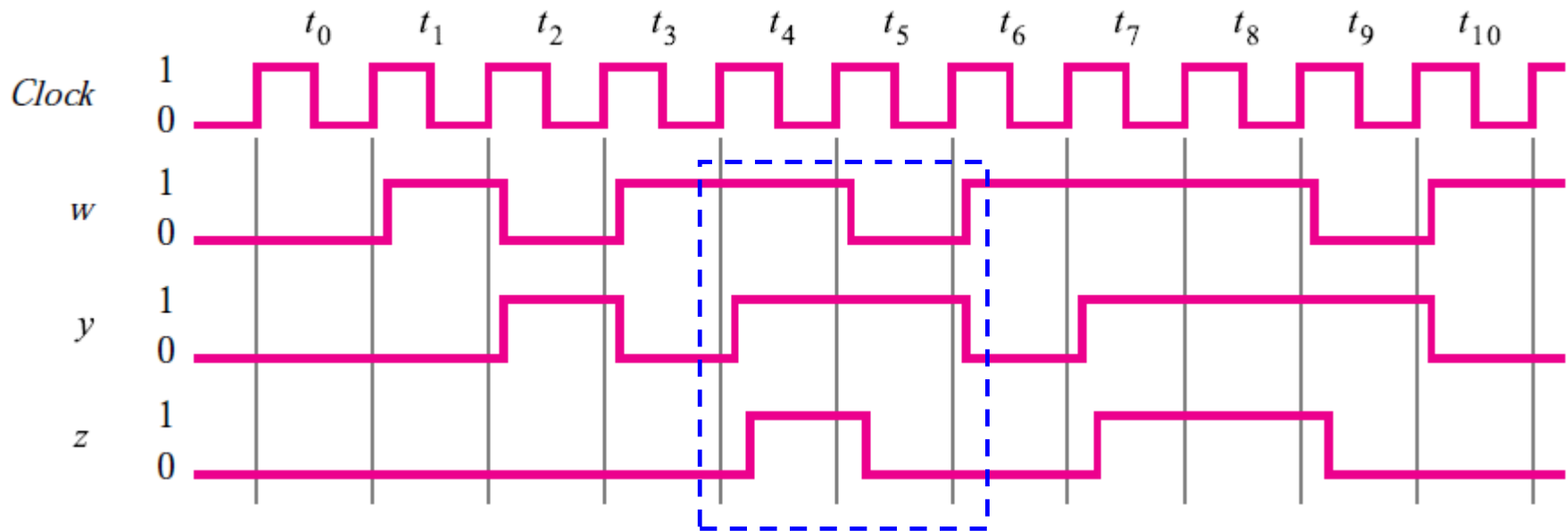
$$z = wy$$





Timing Diagram of Mealy Machine

Notice how output changes based on state and input





State Minimization

- When a designer has to design a more complex FSM, it is likely that the initial attempt will result in a machine that has more states than is actually required
 - Lower # of states $\Rightarrow$ lower # of FFs
- Solved using “partitioning minimization procedure”
 - *Two states S_i and S_j are said to be equivalent if and only if for every possible input sequence, the same output sequence will be produced regardless of whether S_i or S_j is the initial state.*
- Instead of trying to show that some states in a given FSM are equivalent, it is often easier to show that some states are definitely **not** equivalent



Minimization procedure

Suppose that a state machine has a single input w .

If the input signal $w = 0$ is applied to this machine in state S_i and the result is that the machine moves to state S_u , we will say that S_u is a **0-successor** of S_i .

Similarly, if $w = 1$ is applied in the state S_i and it causes the machine to move to state S_v , we will say that S_v is a **1-successor** of S_i .

In general, we will refer to the successors of S_i as its k -successors.

Idea

If the states S_i and S_j are equivalent, then their corresponding k -successors (for all k) are also equivalent



Partitioning

Definition

A partition consists of one or more blocks, where each block comprises a subset of states that may be equivalent, but the states in a given block are definitely not equivalent to the states in other blocks.

● Process

- Let us assume initially that all states are equivalent; this forms the initial partition, P_1 , in which all states are in the same block.
- As the next step, we will form the partition P_2 in which the set of states is partitioned into blocks such that the states in each block generate the same output values.
- Then we will continue to form new partitions...



State Minimization

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	B	C	1
B	D	F	1
C	F	E	0
D	B	G	1
E	F	C	0
F	E	D	0
G	F	G	0

The initial partition contains all states in a single block

$$P_1 = (ABCDEFGG)$$

The next partition separates the states that have different outputs

$$P_2 = (ABD)(CEFG)$$

Now we must consider all 0- and 1-successors of the states in each block.



State Minimization

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	B	C	1
B	D	F	1
C	F	E	0
D	B	G	1
E	F	C	0
F	E	D	0
G	F	G	0

$$P_2 = (ABD)(CEFG)$$

For block (ABD) , the 0-successors are (BDB) . The 1-successors for these states are (CFG) . We treat them as equivalent, no change yet.

For block $(CEFG)$, the 0-successors are $(FFEF)$. The 1-successors are $(EC\textcolor{red}{D}G)$.

Not in the same block in P_2 . Thus,

$$P_3 = (ABD)(CEG)(F)$$



State Minimization

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	B	C	1
B	D	F	1
C	F	E	0
D	B	G	1
E	F	C	0
F	E	D	0
G	F	G	0

$$P_3 = (ABD)(CEG)(F)$$

Partitioning on 0- & 1-successor for blocks (ABD) & (CEG) , we get

$$P_4 = (AD)(B)(CEG)(F)$$

Following the same approach for blocks (AD) & (CEG) , we get

$$P_5 = (AD)(B)(CEG)(F)$$

Since $P_4 = P_5$, no more changes are possible and the states in each block are considered **equivalent**.



State Minimization

Present state	Next state		Output z
	w = 0	w = 1	
A	B	C	1
B	D	F	1
C	F	E	0
D	B	G	1
E	F	C	0
F	E	D	0
G	F	G	0

Present state	Nextstate		Output z
	w = 0	w = 1	
A	B	C	1
B	A	F	1
C	F	C	0
F	C	A	0

Final Partition:

$$P_5 = (AD)(B)(CEG)(F)$$

Only **four** states are needed instead of seven. Hence 2 FFs are sufficient to implement the FSM instead of 3.



Sequential Counter Design



Sequential Counter Design

The specification for the counter is

The counting sequence is $0, 1, 2, \dots, 6, 7, 0, 1, \dots$

There exists an input signal w . The value of this signal is considered during each clock cycle.

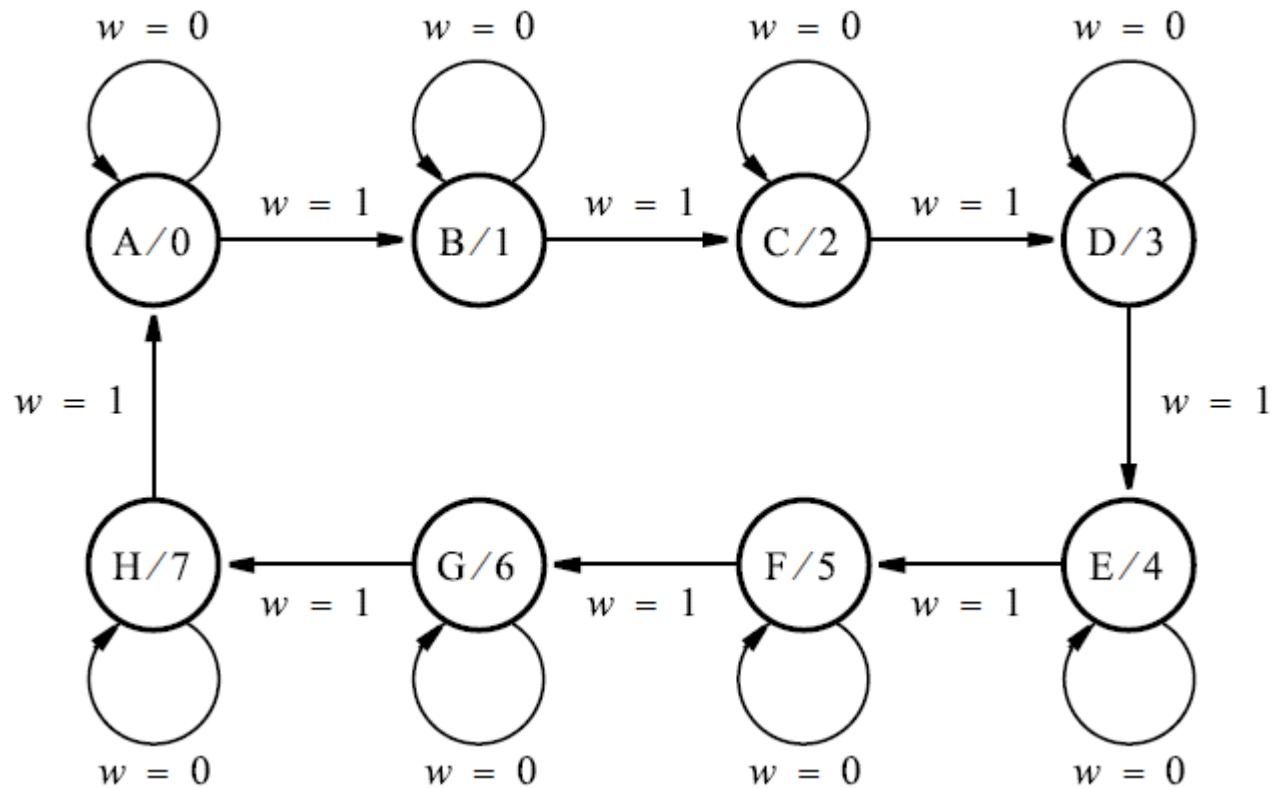
If $w = 0$, the present count remains the same; if $w = 1$, the count is incremented.

The counter can be designed as a synchronous sequential circuit using the design techniques introduced in the previous slides.



State Table

Mod 8 counter (0, 1, .. 7, 0, 1 ..) {Moore Design}





State Assignment

Mod 8 counter (0, 1, .. 7, 0, 1 ..) {Moore Design}

Present state	Next state		Output
	$w = 0$	$w = 1$	
A	A	B	0
B	B	C	1
C	C	D	2
D	D	E	3
E	E	F	4
F	F	G	5
G	G	H	6
H	H	A	7

	Present state $y_2 y_1 y_0$	Next state		Count $z_2 z_1 z_0$
		$w = 0$	$w = 1$	
		$Y_2 Y_1 Y_0$	$Y_2 Y_1 Y_0$	
A	000	000	001	000
B	001	001	010	001
C	010	010	011	010
D	011	011	100	011
E	100	100	101	100
F	101	101	110	101
G	110	110	111	110
H	111	111	000	111



Implementation using D FFs

When using D-type flip-flops to realize the finite state machine, each next-state function, Y_i , is connected to the D input of the flip-flop that implements the state variable y_i .

Using Karnaugh maps, we obtain the following

$$D_0 = Y_0 = \bar{w}y_0 + w\bar{y}_0$$

$$D_1 = Y_1 = \bar{w}y_1 + y_1\bar{y}_0 + wy_0\bar{y}_1$$

$$D_2 = Y_2 = \bar{w}y_2 + \bar{y}_0y_2 + \bar{y}_1y_2 + wy_0y_1\bar{y}_2$$

	Present state $y_2y_1y_0$	Next state		Count $z_2z_1z_0$
		$w = 0$	$w = 1$	
		$Y_2 Y_1 Y_0$	$Y_2 Y_1 Y_0$	
A	000	000	001	000
B	001	001	010	001
C	010	010	011	010
D	011	011	100	011
E	100	100	101	100
F	101	101	110	101
G	110	110	111	110
H	111	111	000	111



Implementation using D FFs

$y_1 y_0$ $w y_2$		00	01	11	10
		00	01	11	10
00		0	1	1	0
01		0	1	1	0
11		1	0	0	1
10		1	0	0	1

$$Y_0 = \bar{w}y_0 + w\bar{y}_0$$

$y_1 y_0$ $w y_2$		00	01	11	10
		00	01	11	10
00		0	0	1	1
01		0	0	1	1
11		0	1	0	1
10		0	1	0	1

$$Y_1 = \bar{w}y_1 + y_1\bar{y}_0 + w y_0\bar{y}_1$$

$y_1 y_0$ $w y_2$		00	01	11	10
		00	01	11	10
00		0	0	0	0
01		1	1	1	1
11		1	1	0	1
10		0	0	1	0

$$Y_2 = \bar{w}y_2 + \bar{y}_0y_2 + \bar{y}_1y_2 + w y_0y_1\bar{y}_2$$

	Present state $y_2 y_1 y_0$	Next state		Count $z_2 z_1 z_0$
		$w = 0$	$w = 1$	
		$Y_2 Y_1 Y_0$	$Y_2 Y_1 Y_0$	
A	000	000	001	000
B	001	001	010	001
C	010	010	011	010
D	011	011	100	011
E	100	100	101	100
F	101	101	110	101
G	110	110	111	110
H	111	111	000	111

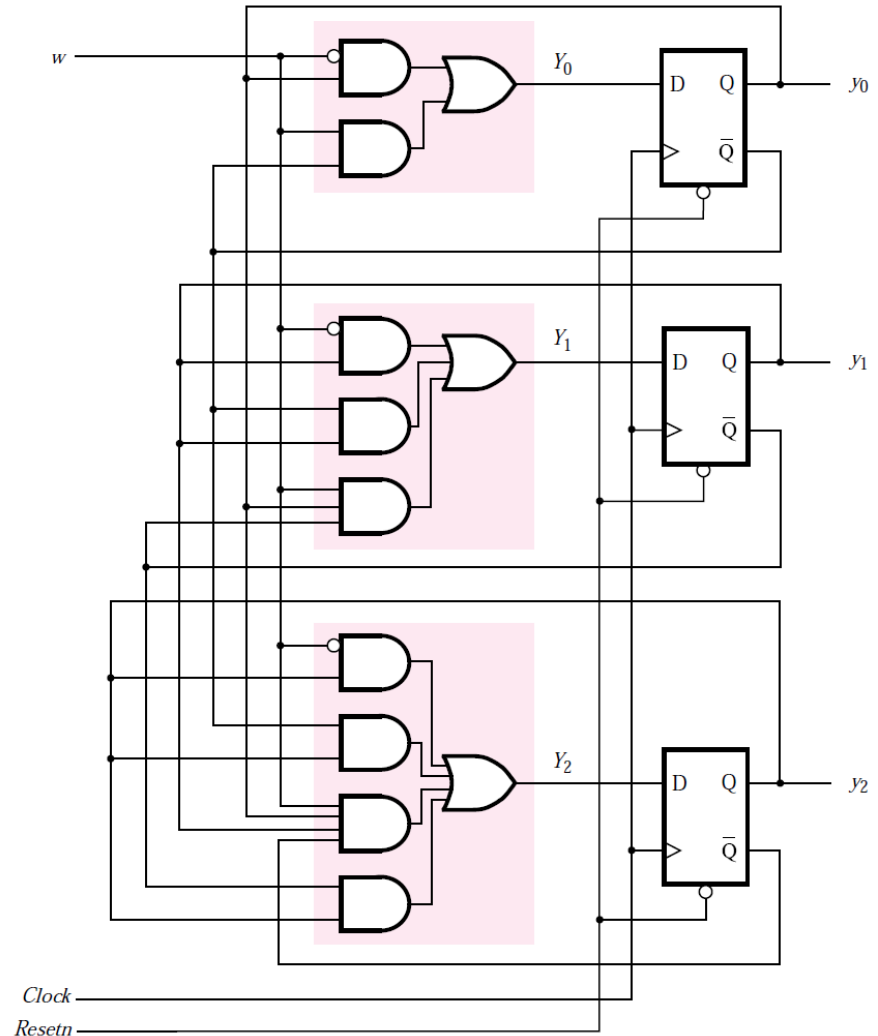


Implementation using D FFs

$$D_0 = Y_0 = \bar{w}y_0 + w\bar{y}_0$$

$$D_1 = Y_1 = \bar{w}y_1 + y_1\bar{y}_0 + wy_0\bar{y}_1$$

$$D_2 = Y_2 = \bar{w}y_2 + \bar{y}_0y_2 + \bar{y}_1y_2 + wy_0y_1\bar{y}_2$$





Sequential Counter Design

Part 2



Sequential Counter Design

Mod 8 counter (0, 1, .. 7, 0, 1 ..) {Moore Design}

Implementation using JK FFs

We must know what values for inputs J and K drive the FF from state $Q(t)$ to state $\bar{Q}(t)$. This information is provided by the **excitation table**.

J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$\bar{Q}(t)$

Characteristic table

$Q(t)$	$Q(t+1)$	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

Excitation table



Excitation table for JK FFs

Mod 8 counter (0, 1, .. 7, 0, 1 ..) {Moore Design}
Implementation using JK FFs

Q(t)	Q(t+1)	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

The excitation table gives the state information in the form of the flip-flop inputs that must be “excited” to cause the transitions to the **desirable** next states.

Present state	Next state		Output
	w = 0	w = 1	
A	A	B	0
B	B	C	1
C	C	D	2
D	D	E	3
E	E	F	4
F	F	G	5
G	G	H	6
H	H	A	7



Sequential Counter Design

Mod 8 counter (0, 1, .. 7, 0, 1 ..) {Moore Design}
Implementation using JK FFs

	Present state $y_2y_1y_0$	Flip-flop inputs								Count $z_2z_1z_0$
		$w = 0$				$w = 1$				
		$Y_2Y_1Y_0$	J_2K_2	J_1K_1	J_0K_0	$Y_2Y_1Y_0$	J_2K_2	J_1K_1	J_0K_0	
A	000	000	0d	0d	0d	001	0d	0d	1d	000
B	001	001	0d	0d	d0	010	0d	1d	d1	001
C	010	010	0d	d0	0d	011	0d	d0	1d	010
D	011	011	0d	d0	d0	100	1d	d1	d1	011
E	100	100	d0	0d	0d	101	d0	0d	1d	100
F	101	101	d0	0d	d0	110	d0	1d	d1	101
G	110	110	d0	d0	0d	111	d0	d0	1d	110
H	111	111	d0	d0	d0	000	d1	d1	d1	111



Sequential Counter Design

Once the table has been derived, it provides a truth table with inputs y_2, y_1, y_0 , and w , and outputs J_2, K_2, J_1, K_1, J_0 , and K_0 .

y_1y_0 wy_2		y_1y_0			
		00	01	11	10
00	0	d	d	0	
01	0	d	d	0	
11	1	d	d	1	
10	1	d	d	1	

$J_0 = w$

y_1y_0 wy_2		y_1y_0			
		00	01	11	10
00	d	0	0	d	
01	d	0	0	d	
11	d	1	1	d	
10	d	1	1	d	

$K_0 = w$

Similarly, we obtain

$$J_0 = K_0 = w$$

$$J_1 = K_1 = wy_0$$

$$J_2 = K_2 = wy_0y_1$$

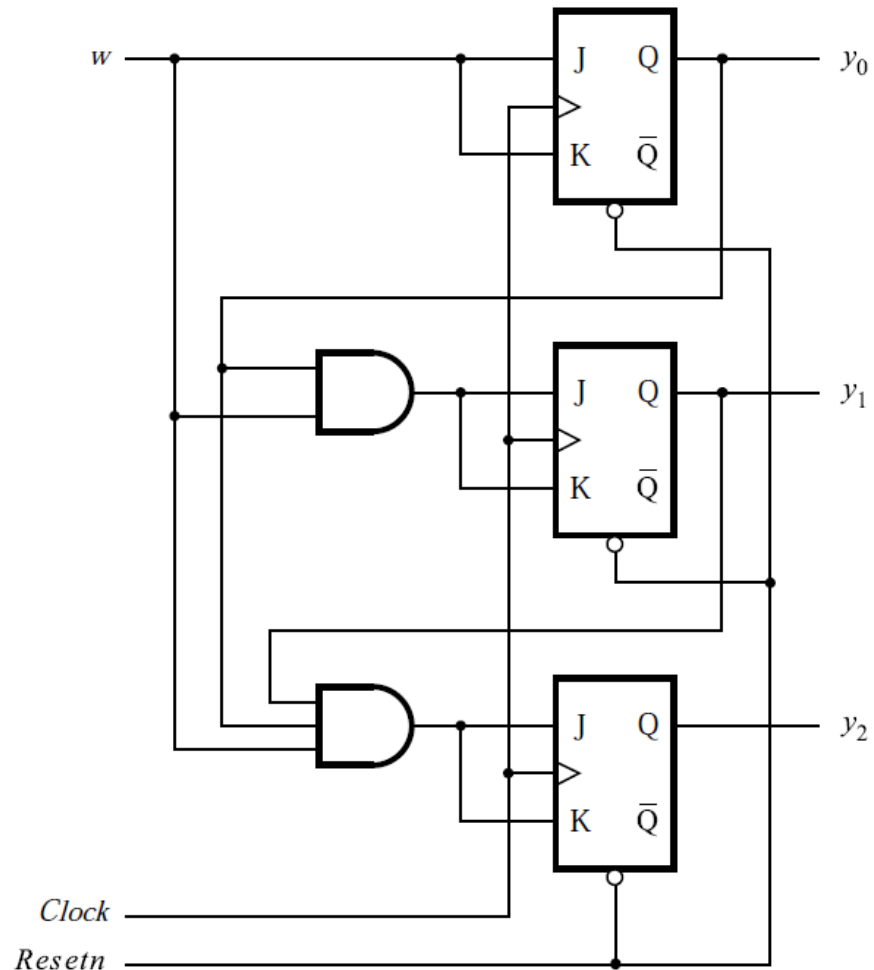


Sequential Counter Design

$$J_0 = K_0 = w$$

$$J_1 = K_1 = wy_0$$

$$J_2 = K_2 = wy_0y_1$$



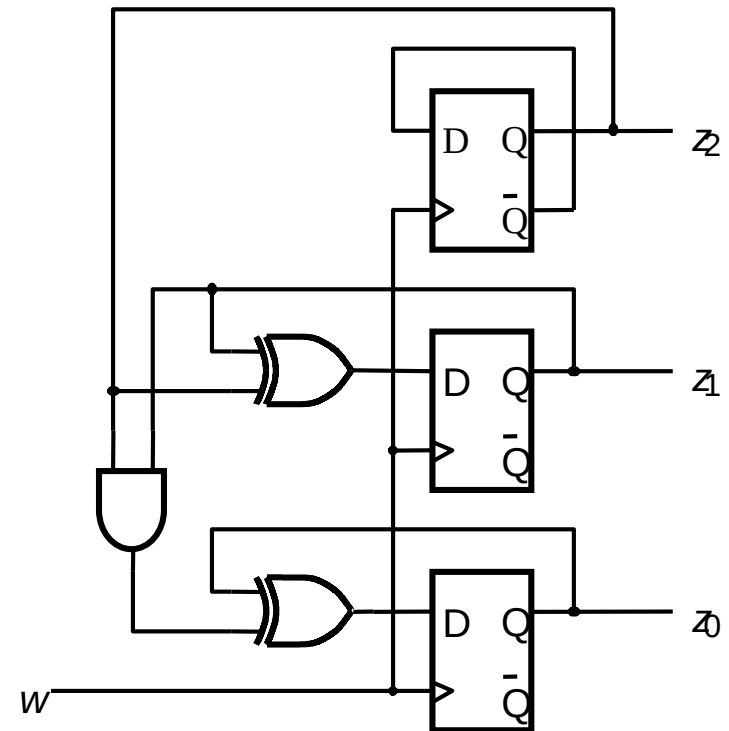


Example: A Different Counter Design

3-bit counter that counts the pulses on an input line w , but display the count in the sequence (0, 4, 2, 6, 1, 5, 3, 7, 0, 4, ..)

Present state	Next state	Output $z_2z_1z_0$
A	B	000
B	C	100
C	D	010
D	E	110
E	F	001
F	G	101
G	H	011
H	A	111

Present state $y_2y_1y_0$	Next state $Y_2Y_1Y_0$	Output $z_2z_1z_0$
000	100	000
100	010	100
010	110	010
110	001	110
001	101	001
101	011	101
011	111	011
111	000	111





Analysis of Synchronous Circuits

Analysis is the reverse of synthesis: Given a circuit, find out what it does.

To analyze a circuit, we simply reverse the steps of the synthesis process. The outputs of the flip-flops represent the present-state variables. Their inputs determine the next state that the circuit will enter.

Steps

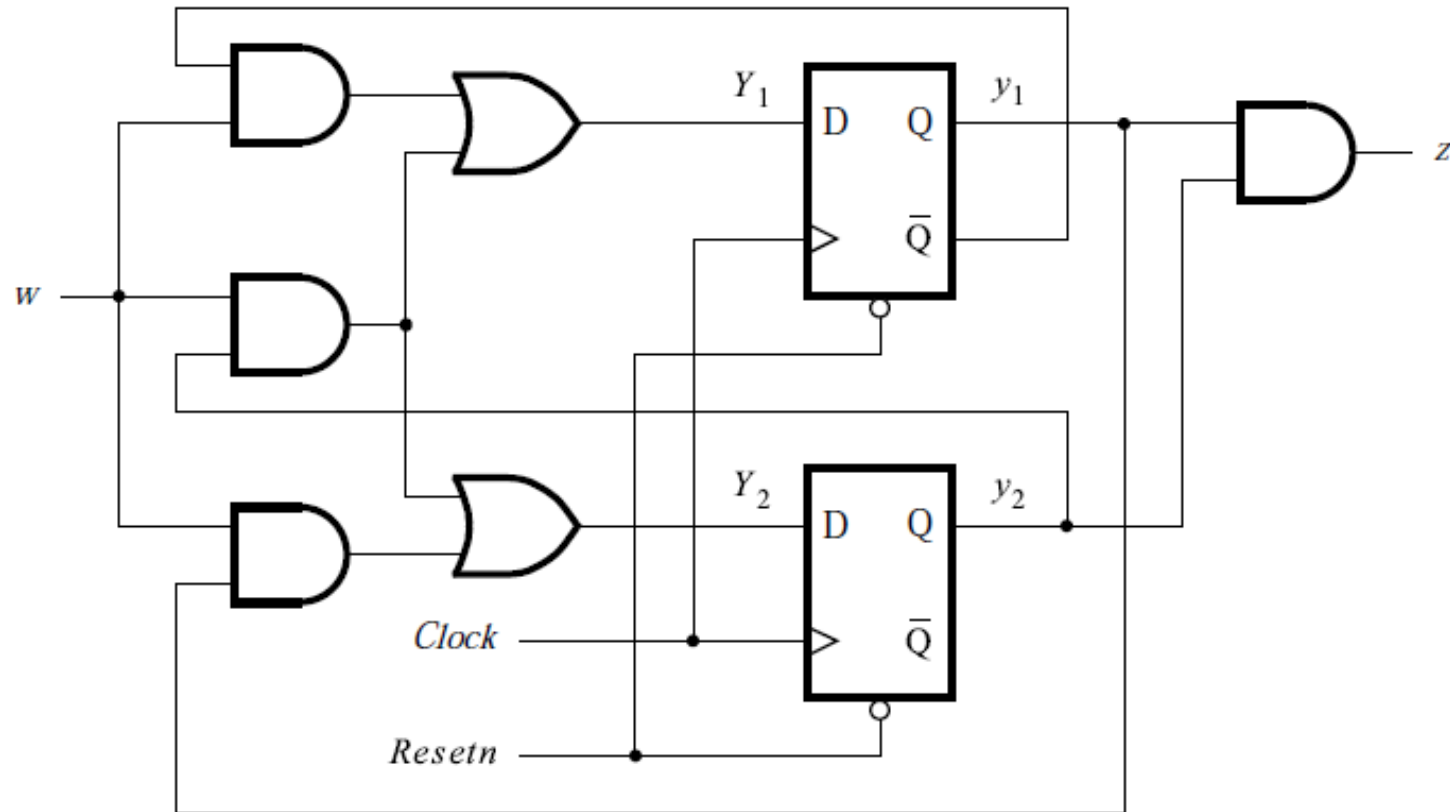
- Construct state table (FFs' type is a factor)

- Symbolic state table

- State diagram



Analysis of Synchronous Circuits



$$Y_1 = wy_1\bar{y_2} + wy_2, \quad Y_2 = wy_1 + wy_2, \quad z = y_1y_2$$



Analysis of Synchronous Circuits

$$\overline{Y}_1 = wy_1 + wy_2, Y_2 = wy_1 + wy_2, z = y_1y_2$$

Present state Y ₂ Y ₁	Next State		Output z
	w = 0	w = 1	
	Y ₂ Y ₁	Y ₂ Y ₁	
0 0	0 0	0 1	0
0 1	0 0	1 0	0
1 0	0 0	1 1	0
1 1	0 0	1 1	1

(a) State-assigned table

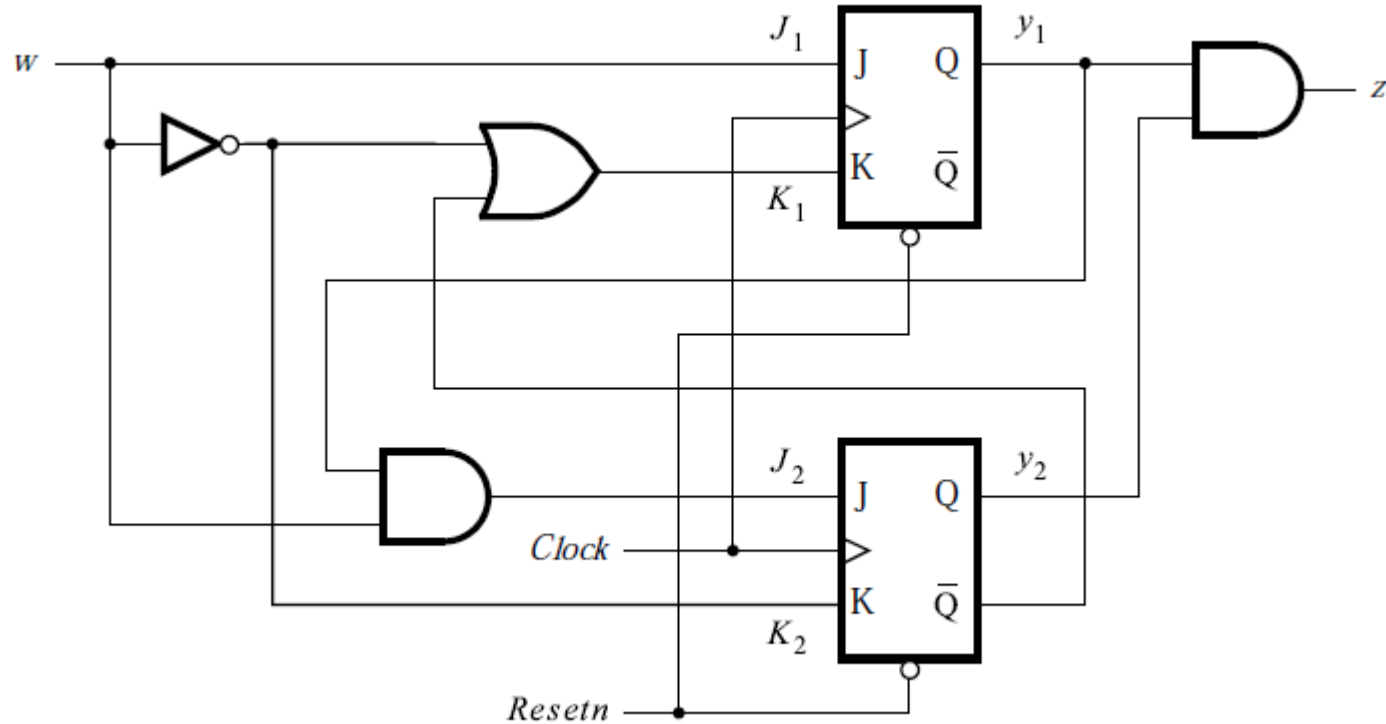
Present state	Next state		Output z
	w = 0	w = 1	
A	A	B	0
B	A	C	0
C	A	D	0
D	A	D	1

(b) State table

Same technique can be applied using other FFs



Analysis of Synchronous Circuits



$$J_1 = w, K_1 = \bar{w} + \bar{y}_2, J_2 = w y_1, K_2 = \bar{w},$$



Analysis of Synchronous Circuits

Present state y_2y_1	Flip-flop inputs				Output z
	$w = 0$		$w = 1$		
	J_2K_2	J_1K_1	J_2K_2	J_1K_1	
00	01	0 1	0 0	1 1	0
01	01	0 1	1 0	1 1	0
10	01	0 1	0 0	1 0	0
11	01	0 1	1 0	1 0	1

$$\begin{aligned}
 J_1 &= w, \\
 K_1 &= \bar{w} + \bar{y}_2, \\
 J_2 &= w y_1, \\
 K_2 &= \bar{w},
 \end{aligned}$$

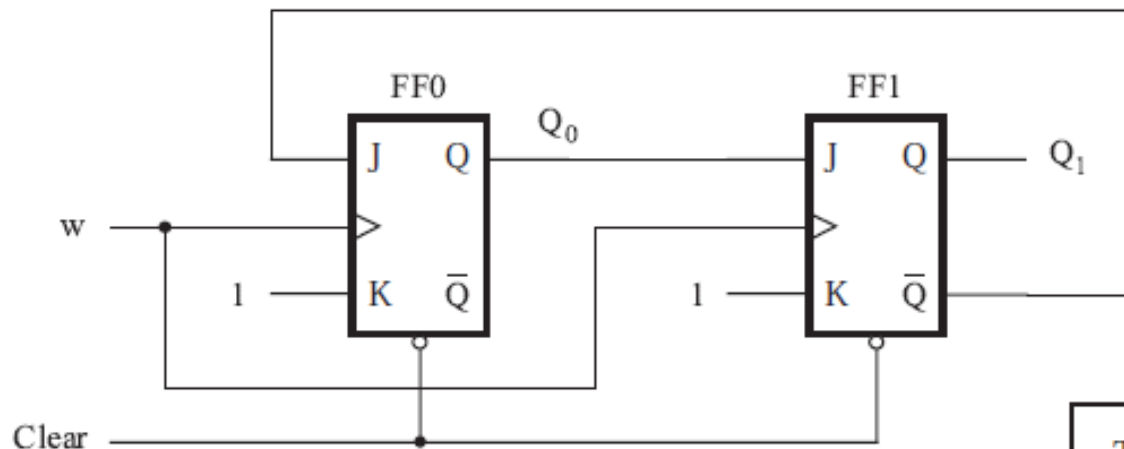
Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	D	0
D	A	D	1

Same can be done using mixed types of FFs (see example 6.11)



Practice: Analysis of Sequential Circuit

Determine the functional behavior of the following circuit. Assume that w is a clock signal and the FFs are cleared first.



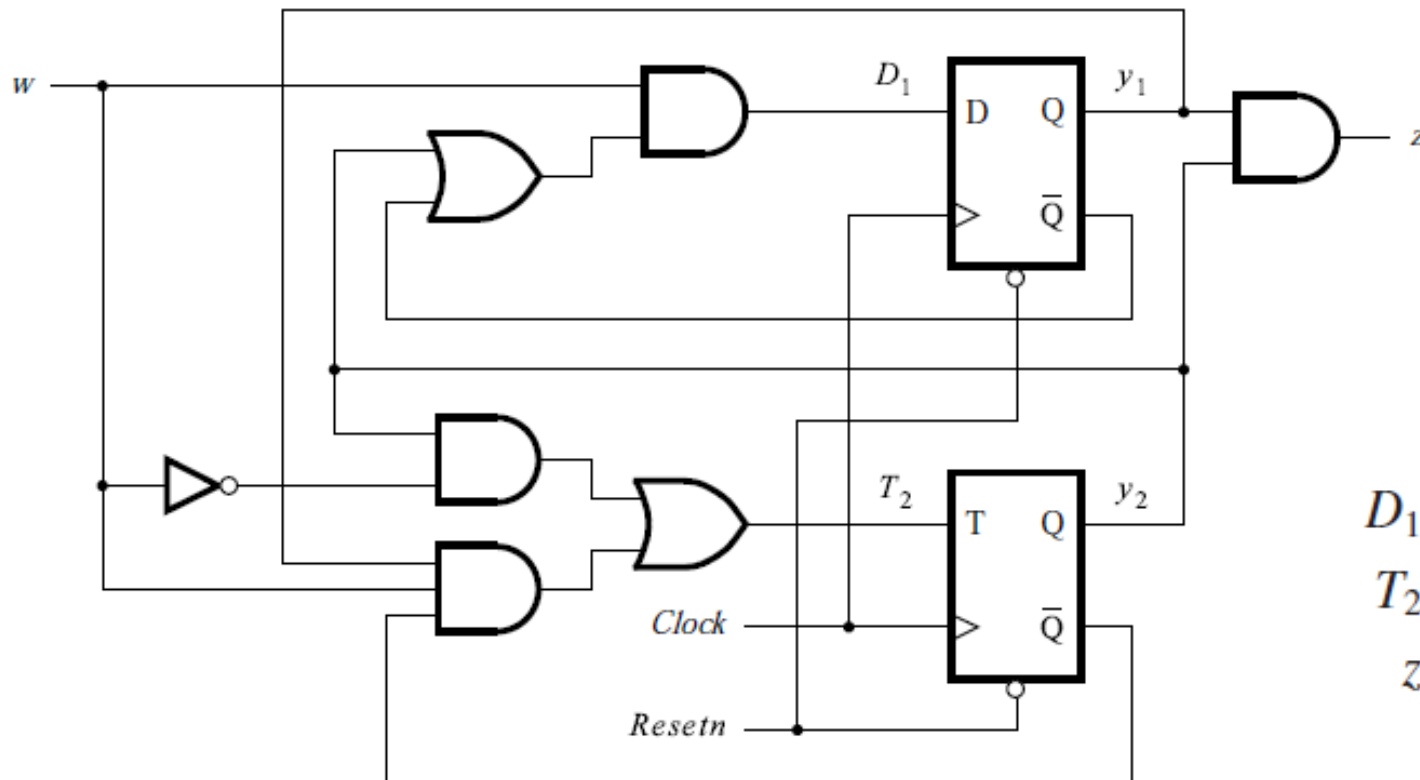
The circuit generates the counting sequence: 0, 1, 2, 0, 1, and so on. Hence, the circuit is a modulo-3 counter.

Time interval	FF0			FF1		
	J_0	K_0	Q_0	J_1	K_1	Q_1
Clear	1	1	0	0	1	0
t_1	1	1	1	1	1	0
t_2	0	1	0	0	1	1
t_3	1	1	0	0	1	0
t_4	1	1	1	1	1	0



Practice: Analysis of Sequential Circuit

Analyze the following circuit, i.e. find the expressions for flip-flop inputs and outputs, and derive the transition table and transition diagrams.

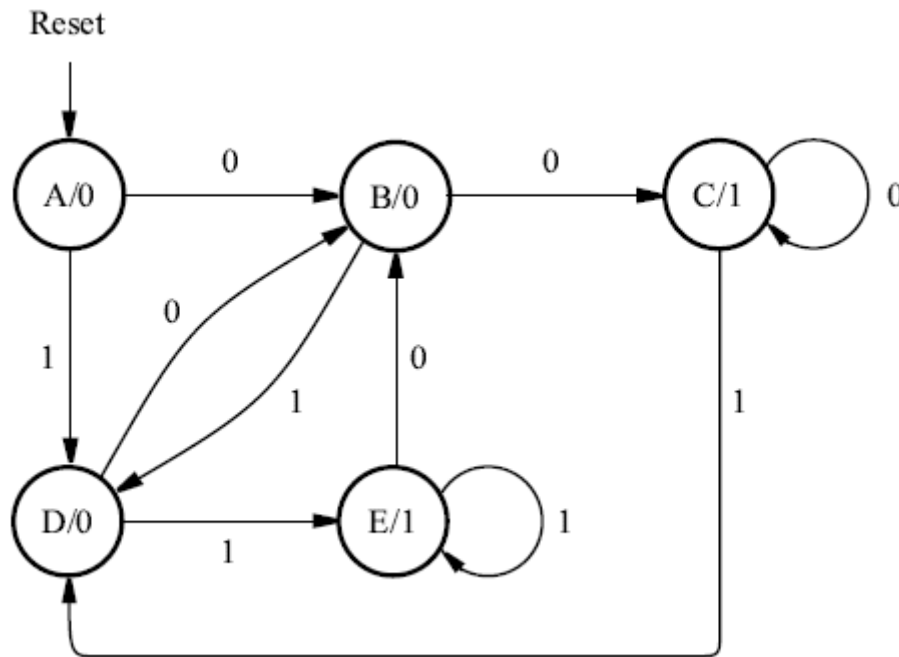


$$\begin{aligned}D_1 &= w(\bar{y}_1 + y_2) \\T_2 &= \bar{w}y_2 + wy_1\bar{y}_2 \\z &= y_1y_2\end{aligned}$$



Examples

Design an FSM that has an input w and an output z . The machine produces $z = 1$ when the previous two values of w were 00 or 11; otherwise $z = 0$.



Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	B	D	0
B	C	D	0
C	C	D	1
D	B	E	0
E	B	E	1



Examples (cont.)

	Present state $y_3y_2y_1$	Next state		Output z
		$w = 0$	$w = 1$	
		$Y_3Y_2Y_1$	$Y_3Y_2Y_1$	
A	000	100	110	0
B	100	101	110	0
C	101	101	110	1
D	110	100	111	0
E	111	100	111	1

$$Y_1 = wy_2 + \bar{w}y_3\bar{y}_2$$

$$Y_2 = w$$

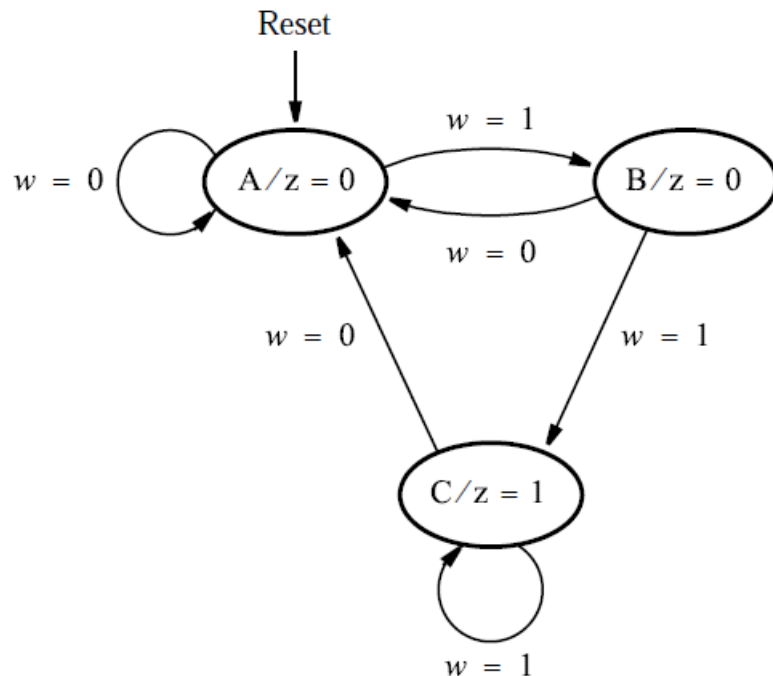
$$Y_3 = 1$$

$$z = y_1$$



Examples (cont.)

Implement the sequence detector of the previous example by using two FSMs. One FSM detects the occurrence of consecutive 1s, while the other detects consecutive 0s.



Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

Diagram for 2 consecutive ones



Examples (cont.)

Two consecutive ones

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

Present state $y_2 y_1$	Next state		Output z
	$w = 0$	$w = 1$	
	$Y_2 Y_1$	$Y_2 Y_1$	
A 00	00	01	0
B 01	00	11	0
C 11	00	11	1
	dd	dd	d

$$Y_1 = w$$

$$Y_2 = wy_1$$

$$z_{ones} = y_2$$

$$z = z_{ones} + z_{zeros}$$

Two consecutive zeros

Present state	Next state		Output z_{zeros}
	$w = 0$	$w = 1$	
D	E	D	0
E	F	D	0
F	F	D	1

Present state $y_4 y_3$	Next state		Output z_{zeros}
	$w = 0$	$w = 1$	
	$Y_4 Y_3$	$Y_4 Y_3$	
D 00	01	00	0
E 01	11	00	0
F 11	11	00	1
	dd	dd	d

$$Y_3 = \bar{w}$$

$$Y_4 = \bar{w}y_3$$

$$z_{zeros} = y_4$$



Examples (cont.)

Design a counter such that when the input $z=1$, the counter alternates between states $10 \rightarrow 01 \rightarrow 10 \rightarrow 01 \dots$. When the input $z=0$, the counter maintains its state. Study the effect of unused states and make sure the counter is **self-correcting**.

If for any reason, the circuit finds itself in one of its unused states, it is necessary to ensure that the circuit eventually goes into one of its **valid** states.

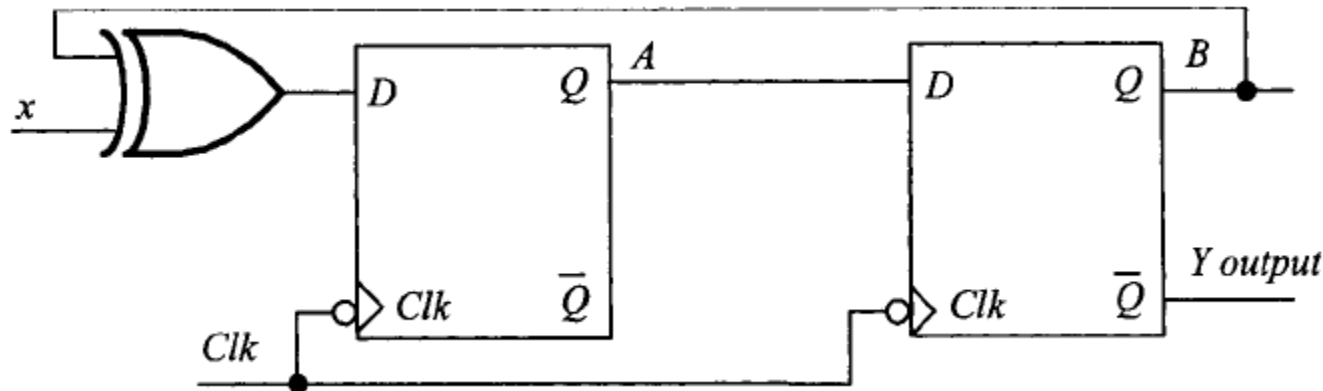
Otherwise, if it circulates among invalid states, there will be no way to bring it back to its intended sequence of state transitions.



Examples (cont.)

Analyze the clocked synchronous circuit shown below. Express the next state in terms of the present state and inputs, derive the state table, and draw the state diagram.

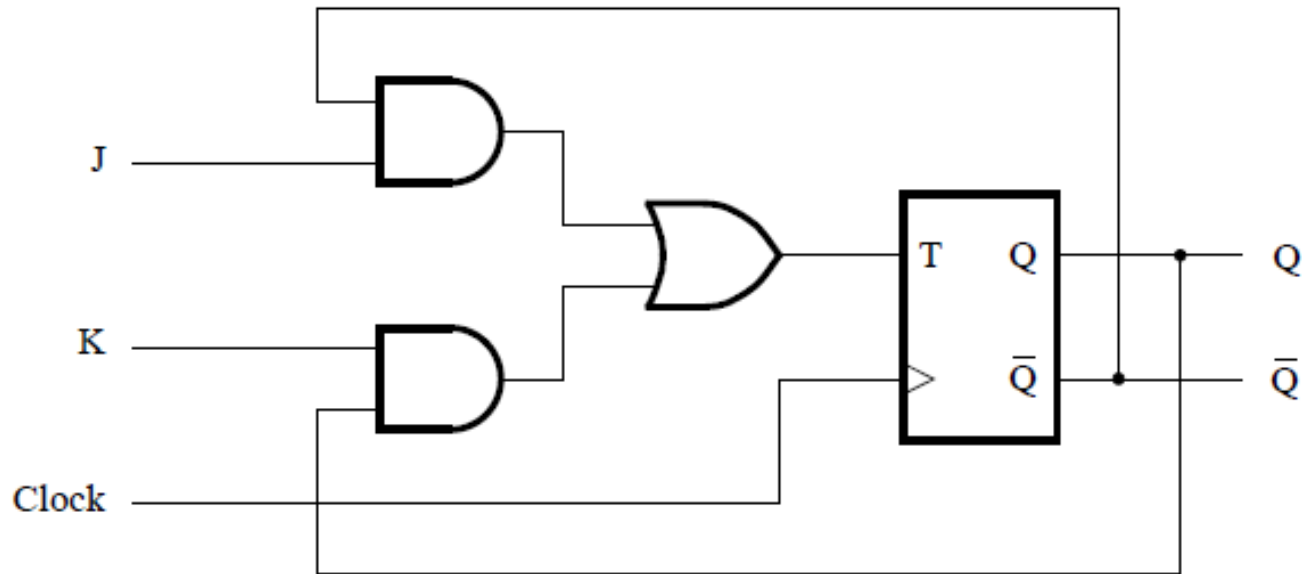
Implement the same circuit using JK FFs.





Examples (cont.)

Implement a JK FF using a T flip-flop.





Practice: Design of Sequential Circuit

A sequential circuit has two inputs, w_1 and w_2 , and an output, z . Its function is to compare the input sequences on the two inputs. If $w_1 = w_2$ during any four consecutive clock cycles, the circuit produces $z = 1$; otherwise, $z = 0$. For example

$w_1 : 0110111000110$

$w_2 : 1110101000111$

$z : 0000100001110$

- Derive a suitable circuit.